

DATABASE MANAGEMENT SYSTEM

MINI PROJECT REPORT

COLLEGE EVENT MANAGEMENT SYSTEM

DBMS MICRO PROJECT — MYSQL

College Format Practical / Mini Project

Made By : Tanmay Zade(153)

Soham Bhishma(147)

INDEX

Sr.	Contents	Page
1	Project Structure / 19-Step Sequence	3
2	Aim, Objectives, Theory & Requirements	4
3	Original MySQL Code	5–11
4	Basic SQL Query Outputs	12
5	JOIN Outputs	13
6	Aggregate Functions	14
7	GROUP BY & HAVING	14
8	Subquery	14
9	View, Index & Trigger	15
10	CRUD — Create & Read	16
11	CRUD — Update & Delete	16
12	Fronted App/Webpage Dashboard Screenshots	17-20
13	Result & Conclusion	21
14	Advantages, Disadvantages & Future Scope	21

The newly supplied SQL is used as the source. The CRUD section contains Create, Read, Update and Delete operations.

PROJECT STRUCTURE / 19-STEP SEQUENCE

Step	Module
1	Create Student Table
2	Create Organizer Table
3	Create Event Table
4	Create Registration Table
5	Create Attendance Table
6	Insert Student Data
7	Insert Organizer Data
8	Insert Event Data
9	Insert Registration Data
10	Insert Attendance Data
11	Basic SQL Queries
12	JOIN Queries
13	Aggregate Functions
14	GROUP BY and HAVING
15	Subquery
16	View
17	Index
18	Trigger
19	CRUD Operations — Create, Read, Update, Delete
20	Fronted App/Webpage Dashboard Screenshots

1. AIM

To design and implement a College Event Management System using MySQL and perform database operations including table creation, insertion, retrieval, joins, aggregate functions, grouping, subqueries, views, indexes, triggers and complete CRUD operations.

2. OBJECTIVES

- Create and manage relational tables for students, organizers, events, registrations and attendance.
- Apply primary keys, foreign keys, unique constraints and relationships.
- Execute SQL queries, joins, aggregate functions, GROUP BY, HAVING and subqueries.
- Implement views, indexes and triggers.
- Demonstrate Create, Read, Update and Delete operations.

3. THEORY

A database management system stores and organizes related data so it can be efficiently inserted, updated, searched and maintained. The College Event Management System uses related tables to manage students, organizers, events, registrations and attendance. SQL is used for DDL and DML operations, joins and data analysis, while views, indexes and triggers provide additional database functionality.

4. SOFTWARE / HARDWARE REQUIREMENTS

Software: MySQL / MySQL Workbench or compatible MySQL compiler.

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard, mouse and sufficient storage.

5. ORIGINAL MYSQL CODE — RESET AND TABLES

Reset

```
DROP TRIGGER IF EXISTS
after_registration; DROP VIEW IF
EXISTS Student_Event_View;
```

```
DROP TABLE IF EXISTS
Attendance;
DROP TABLE IF EXISTS
Registration;
DROP TABLE IF EXISTS
`Event`;
DROP TABLE IF EXISTS
Organizer;
DROP TABLE IF EXISTS
Student;
```

```
/* =====
1. CREATE STUDENT TABLE
===== */
```

Student

```
CREATE TABLE
Student (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50) NOT
    NULL, Branch VARCHAR(30),
    `Year` INT,
    Email VARCHAR(100) UNIQUE
);
```

```
/* =====
2. CREATE ORGANIZER TABLE
===== */
```

Organizer

```
CREATE TABLE
Organizer (
    Organizer_ID INT PRIMARY
    KEY, Organizer_Name
    VARCHAR(50) NOT NULL,
    Department VARCHAR(50),
    Contact VARCHAR(15)
);
```

```
/* =====
3. CREATE EVENT TABLE
===== */
```

5. ORIGINAL MYSQL CODE — EVENT AND REGISTRATION

Event

```
CREATE TABLE
`Event` (
  Event_ID INT PRIMARY KEY,
  Event_Name VARCHAR(100)
  NOT NULL,
  Event_Date
  DATE, Venue
  VARCHAR(100),
  Organizer_ID
  INT,
  Max_Seats
  INT,

  FOREIGN KEY (Organizer_ID)
  REFERENCES
  Organizer(Organizer_ID)
);

/* =====
4. CREATE REGISTRATION TABLE
===== */
```

Registration

```
CREATE TABLE
Registration (
  Registration_ID INT PRIMARY
  KEY, Student_ID INT,
  Event_ID INT,
  Registration_Date
  DATE,

  FOREIGN KEY (Student_ID)
  REFERENCES
  Student(Student_ID),

  FOREIGN KEY (Event_ID)
  REFERENCES `Event`(Event_ID),

  UNIQUE(Student_ID, Event_ID)
);

/* =====
5. CREATE ATTENDANCE TABLE
===== */
```

Attendance

```
CREATE TABLE
Attendance (
  Attendance_ID INT
  PRIMARY KEY,
  Registration_ID INT
  UNIQUE, Status
  VARCHAR(20),

  FOREIGN KEY (Registration_ID)
  REFERENCES Registration(Registration_ID)
);
```

5. ORIGINAL MYSQL CODE — INSERT DATA

Student Data

```
INSERT INTO Student
(Student_ID, Student_Name, Branch,
`Year`, Email) VALUES
(101, 'Amit Sharma', 'IT', 2, 'amit@gmail.com'),
(102, 'Rahul Patil', 'CSE', 2, 'rahul@gmail.com'),
(103, 'Priya Singh', 'IT', 3, 'priya@gmail.com'),
(104, 'Sneha Joshi', 'ENTC', 2, 'sneha@gmail.com'),
(105, 'Rohan Verma', 'CSE', 3, 'rohan@gmail.com');

/* =====
7. INSERT ORGANIZER DATA
===== */
```

Organizer Data

```
INSERT INTO Organizer
(Organizer_ID, Organizer_Name,
Department, Contact) VALUES
(1, 'Tech Club', 'Information Technology', '9876543210'),
(2, 'Cultural Club', 'Arts', '9876543211'),
(3, 'Sports Club', 'Physical Education', '9876543212');

/* =====
8. INSERT EVENT DATA
===== */
```

Event Data

```
INSERT INTO `Event`
(Event_ID, Event_Name, Event_Date, Venue,
Organizer_ID, Max_Seats) VALUES
(201, 'Coding Competition', '2026-10-10', 'Computer Lab', 1, 50),
(202, 'Dance Competition', '2026-10-15', 'Auditorium', 2, 100),
(203, 'Cricket Tournament', '2026-10-20', 'College Ground', 3, 80),
(204, 'Quiz Competition', '2026-10-25', 'Seminar Hall', 1, 60);

/* =====
9. INSERT REGISTRATION DATA
===== */
```

Registration Data

```
INSERT INTO Registration
(Registration_ID, Student_ID, Event_ID,
Registration_Date) VALUES
(301, 101, 201, '2026-09-20'),
(302, 102, 201, '2026-09-21'),
(303, 103, 202, '2026-09-22'),
(304, 104, 203, '2026-09-23'),
(305, 105, 204, '2026-09-24');

/* =====
10. INSERT ATTENDANCE DATA
===== */
```

Attendance Data

```
10. INSERT ATTENDANCE DATA
===== */

INSERT INTO Attendance
(Attendance_ID, Registration_ID,
Status) VALUES
(401, 301, 'Present'),
(402, 302, 'Present'),
(403, 303, 'Absent'),
(404, 304, 'Present'),
(405, 305, 'Present');

/* =====
```

5. ORIGINAL MYSQL CODE — BASIC SQL AND JOINS

Basic Queries

```
11. BASIC SQL QUERIES
===== */

/* Display all students
*/ SELECT *
FROM Student;

/* Display all
events */ SELECT *
FROM `Event`;

/* Display students from IT
branch */ SELECT *
FROM Student
WHERE Branch = 'IT';

/* Display events conducted in
Auditorium */ SELECT *
FROM `Event`
WHERE Venue = 'Auditorium';

/* Update student
information */ UPDATE
Student
SET Email =
'amit.sharma@gmail.com' WHERE
Student_ID = 101;

/* Display updated student
*/ SELECT *
FROM Student
WHERE Student_ID = 101;

/* =====
```

JOIN Queries

```
12. JOIN QUERIES
===== */

/* Student names and registered events */

SELECT
    Student.Student_Name,
    `Event`.Event_Name
FROM Registration
INNER JOIN Student
    ON Registration.Student_ID = Student.Student_ID
INNER JOIN `Event`
    ON Registration.Event_ID = `Event`.Event_ID;

/* Event and organizer details */

SELECT
    `Event`.Event_Name,
    Organizer.Organizer_Name,
    Organizer.Department
FROM `Event`
INNER JOIN Organizer
    ON `Event`.Organizer_ID = Organizer.Organizer_ID;

/* Complete registration information */

SELECT
    Registration.Registration_ID,
    Student.Student_Name,
    `Event`.Event_Name,
    Registration.Registration_Date
FROM Registration
INNER JOIN Student
    ON Registration.Student_ID = Student.Student_ID
INNER JOIN `Event`
    ON Registration.Event_ID = `Event`.Event_ID;

/* =====
```


5. ORIGINAL MYSQL CODE — AGGREGATES AND GROUPING

Aggregate Functions

```
13. AGGREGATE FUNCTIONS
===== */

/* Count total students */

SELECT COUNT(*) AS
Total_Students FROM
Student;

/* Count total events */

SELECT COUNT(*) AS
Total_Events FROM `Event`;

/* Count registrations for each
event */
SELECT
    `Event`.Event_Name,
    COUNT(Registration.Registration_ID) AS
Total_Registrations FROM `Event`
LEFT JOIN Registration
    ON `Event`.Event_ID =
Registration.Event_ID GROUP BY
    `Event`.Event_ID,
    `Event`.Event_Name;

/* =====
```

GROUP BY and HAVING

```
14. GROUP BY AND HAVING
===== */

SELECT
    `Event`.Event_Name,
    COUNT(Registration.Registration_ID) AS
Total_Students
FROM `Event`
INNER JOIN Registration
    ON `Event`.Event_ID =
Registration.Event_ID GROUP BY
    `Event`.Event_ID,
    `Event`.Event_Name
HAVING COUNT(Registration.Registration_ID) > 1;

/* =====
```

Subquery

```
15. SUBQUERY
===== */

SELECT
Student_Name
FROM Student
WHERE
Student_ID IN
(
    SELECT
    Student_ID
    FROM
    Registration
    WHERE
    Event_ID = (
        SELECT
        Event_ID
        FROM
        `Event`
        WHERE Event_Name = 'Coding Competition'
    )
);

/* =====
```

5. ORIGINAL MYSQL CODE — VIEW, INDEX AND TRIGGER

View

```
16. VIEW
===== */
CREATE VIEW
Student_Event_View AS
SELECT
    Student.Student_Name,
    Student.Branch,
    `Event`.Event_Name,
    `Event`.Event_Date,
    `Event`.Ven
ue FROM
Registration
INNER JOIN
Student
    ON Registration.Student_ID =
Student.Student_ID INNER JOIN `Event`
    ON Registration.Event_ID = `Event`.Event_ID;

/* Display View

*/ SELECT *
FROM Student_Event_View;

/* =====
```

Index

```
17. INDEX
===== */
CREATE INDEX
idx_event_name ON
`Event`(Event_Name);

/* Display Index

Information */ SHOW INDEX

FROM `Event`;

/* =====
```

Trigger

```
18. TRIGGER
===== */

DELIMITER //

CREATE TRIGGER after_registration
AFTER INSERT ON Registration
FOR EACH ROW
BEGIN

    INSERT INTO Attendance
    (Attendance_ID, Registration_ID, Status)
    VALUES
    (NEW.Registration_ID + 100,
    NEW.Registration_ID,
    'Pending');

END //

DELIMITER ;

/* Test Trigger */

INSERT INTO Registration
(Registration_ID, Student_ID, Event_ID, Registration_Date)
VALUES
(306, 101, 204, '2026-09-25');

/* Display Attendance after Trigger */

SELECT *
FROM Attendance;

/* =====
```

5. ORIGINAL MYSQL CODE — CRUD OPERATIONS

CRUD

19. CRUD OPERATIONS

```
===== */

/*----- CREATE-----
        */ INSERT INTO

Student
(Student_ID, Student_Name, Branch,
`Year`, Email) VALUES
(106, 'Karan Mehta', 'IT', 2, 'karan@gmail.com');

/*----- READ-----
        */ SELECT *
FROM Student;

/*----- UPDATE-----
        */ UPDATE Student
SET Branch =
'CSE' WHERE
Student_ID = 106;

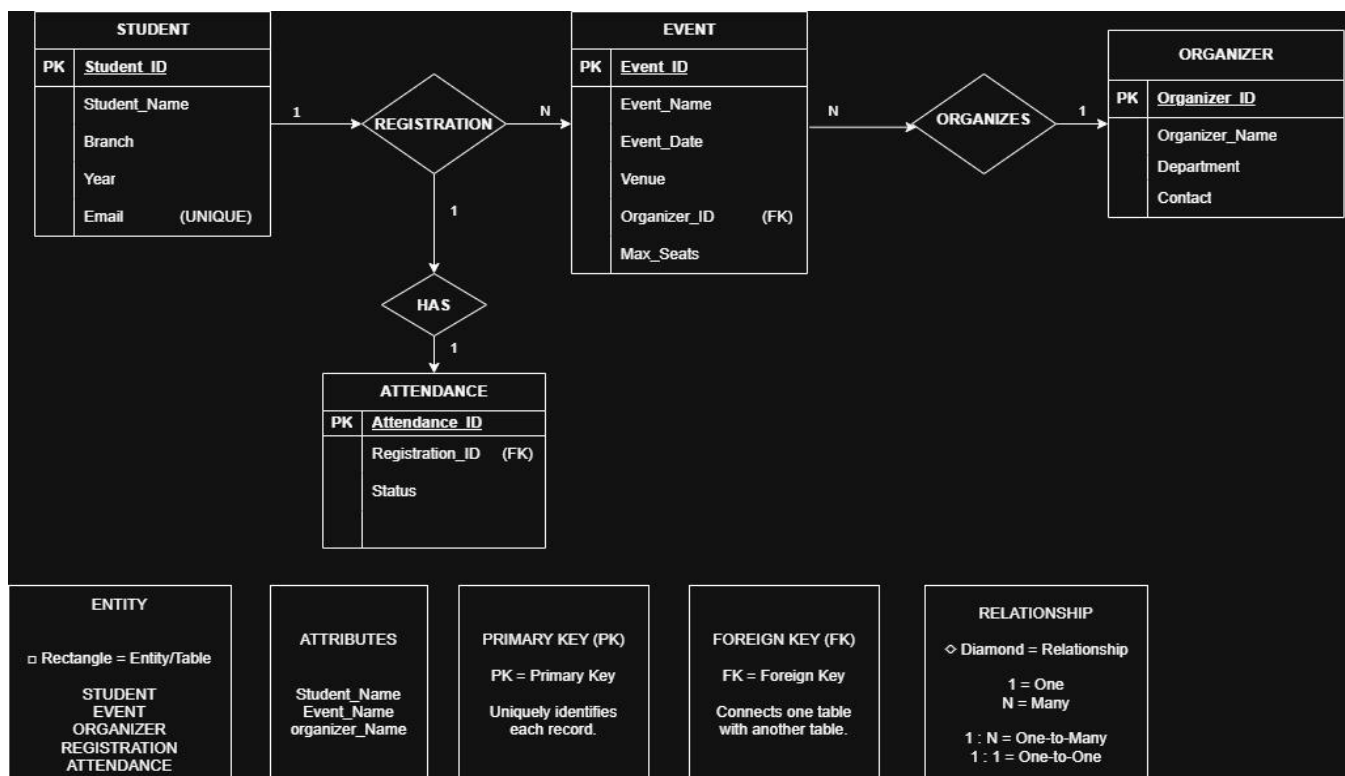
/* Check updated record
*/ SELECT *
FROM Student
WHERE Student_ID = 106;

/*----- DELETE-----
        */ DELETE FROM

Student
WHERE Student_ID = 106;

/* Check deleted record
*/ SELECT *
FROM Student;
```

ER MODEL DIAGRAM :



6. OUTPUT — BASIC SQL QUERIES

Display all students

Student_ID	Student_Name	Branch	Year	Email
101	Amit Sharma	IT	2	amit.sharma@gmail.com
102	Rahul Patil	CSE	2	rahul@gmail.com
103	Priya Singh	IT	3	priya@gmail.com
104	Sneha Joshi	ENTC	2	sneha@gmail.com
105	Rohan Verma	CSE	3	rohan@gmail.com

Students from IT branch

Student_ID	Student_Name	Branch	Year	Email
101	Amit Sharma	IT	2	amit.sharma@gmail.com
103	Priya Singh	IT	3	priya@gmail.com

Events conducted in Auditorium

Event_ID	Event_Name	Event_Date	Venue	Organizer_ID	Max_Seats
202	Dance Competition	2026-10-15	Auditorium	2	100

Updated student 101

Student_ID	Student_Name	Branch	Year	Email
101	Amit Sharma	IT	2	amit.sharma@gmail.com

7. OUTPUT — JOIN QUERIES

Student names and registered events

Student_Name	Event_Name
Amit Sharma	Coding Competition
Rahul Patil	Coding Competition
Priya Singh	Dance Competition
Sneha Joshi	Cricket Tournament
Rohan Verma	Quiz Competition

Event and organizer details

Event_Name	Organizer_Name	Department
Coding Competition	Tech Club	Information Technology
Dance Competition	Cultural Club	Arts
Cricket Tournament	Sports Club	Physical Education
Quiz Competition	Tech Club	Information Technology

Complete registration information

Registration_ID	Student_Name	Event_Name	Registration_Date
301	Amit Sharma	Coding Competition	2026-09-20
302	Rahul Patil	Coding Competition	2026-09-21
303	Priya Singh	Dance Competition	2026-09-22
304	Sneha Joshi	Cricket Tournament	2026-09-23
305	Rohan Verma	Quiz Competition	2026-09-24

8. OUTPUT — AGGREGATE, GROUP BY AND SUBQUERY

Total students

Total_Students
5

Total events

Total_Events
4

Registrations for each event

Event_Name	Total_Registrations
Coding Competition	2
Dance Competition	1
Cricket Tournament	1
Quiz Competition	1

GROUP BY and HAVING

Event_Name	Total_Students
Coding Competition	2

Subquery: Coding Competition students

Student_Name
Amit Sharma
Rahul Patil

9. OUTPUT — VIEW, INDEX AND TRIGGER

Student_Event_View

Student_Name	Branch	Event_Name	Event_Date	Venue
Amit Sharma	IT	Coding Competition	2026-10-10	Computer Lab
Rahul Patil	CSE	Coding Competition	2026-10-10	Computer Lab
Priya Singh	IT	Dance Competition	2026-10-15	Auditorium
Sneha Joshi	ENTC	Cricket Tournament	2026-10-20	College Ground
Rohan Verma	CSE	Quiz Competition	2026-10-25	Seminar Hall

Index information

Table	Non_unique	Key_name	Column_name	Index_type
Event	0	PRIMARY	Event_ID	BTREE
Event	1	idx_event_name	Event_Name	BTREE

Attendance after Trigger

Attendance_ID	Registration_ID	Status
401	301	Present
402	302	Present
403	303	Absent
404	304	Present
405	305	Present
406	306	Pending

10. CRUD OUTPUT — CREATE AND READ

CREATE: Insert Karan Mehta

Student_ID	Student_Name	Branch	Year	Email
106	Karan Mehta	IT	2	karan@gmail.com

READ: Student table after CREATE

Student_ID	Student_Name	Branch	Year	Email
101	Amit Sharma	IT	2	amit.sharma@gmail.com
102	Rahul Patil	CSE	2	rahul@gmail.com
103	Priya Singh	IT	3	priya@gmail.com
104	Sneha Joshi	ENTC	2	sneha@gmail.com
105	Rohan Verma	CSE	3	rohan@gmail.com
106	Karan Mehta	IT	2	karan@gmail.com

11. CRUD OUTPUT — UPDATE AND DELETE

UPDATE: Branch of Student 106 changed from IT to CSE

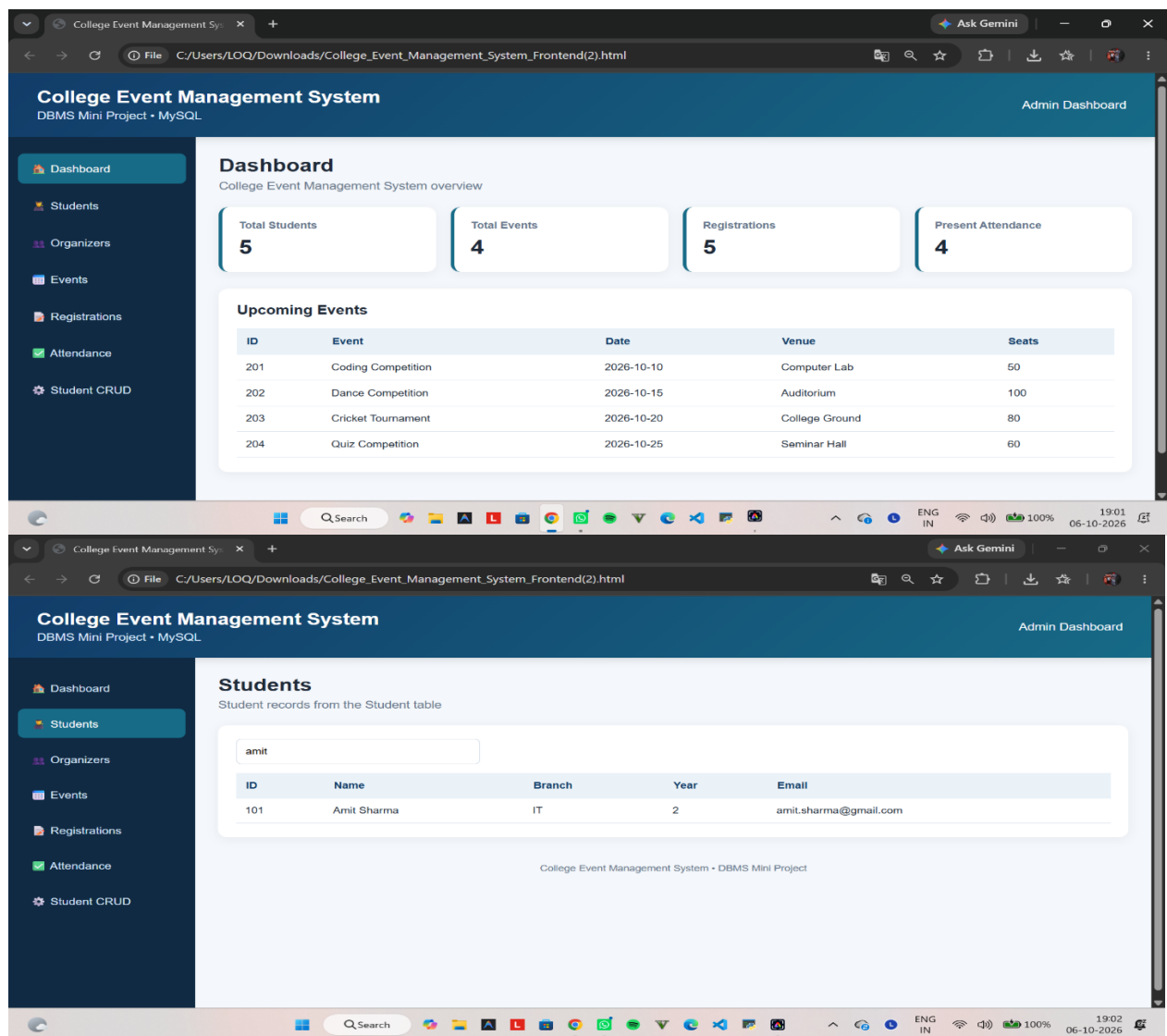
Student_ID	Student_Name	Branch	Year	Email
106	Karan Mehta	CSE	2	karan@gmail.com

DELETE: Student 106 removed

Student_ID	Student_Name	Branch	Year	Email
101	Amit Sharma	IT	2	amit.sharma@gmail.com
102	Rahul Patil	CSE	2	rahul@gmail.com
103	Priya Singh	IT	3	priya@gmail.com
104	Sneha Joshi	ENTC	2	sneha@gmail.com
105	Rohan Verma	CSE	3	rohan@gmail.com

The final SELECT confirms that Student_ID 106 is absent after DELETE.

12. Frontend Webpage screenshots:



College Event Management System

DBMS Mini Project • MySQL

Admin Dashboard

Dashboard

Students

Organizers

Events

Registrations

Attendance

Student CRUD

Organizers

Organizer details

ID	Organizer	Department	Contact
1	Tech Club	Information Technology	9876543210
2	Cultural Club	Arts	9876543211
3	Sports Club	Physical Education	9876543212

College Event Management System • DBMS Mini Project

College Event Management System

DBMS Mini Project • MySQL

Admin Dashboard

Dashboard

Students

Organizers

Events

Registrations

Attendance

Student CRUD

Events

College events stored in the Event table

ID	Event Name	Date	Venue	Organizer ID	Max Seats
201	Coding Competition	2026-10-10	Computer Lab	1	50
202	Dance Competition	2026-10-15	Auditorium	2	100
203	Cricket Tournament	2026-10-20	College Ground	3	80
204	Quiz Competition	2026-10-25	Seminar Hall	1	60

College Event Management System • DBMS Mini Project

College Event Management System

DBMS Mini Project • MySQL

Admin Dashboard

Dashboard

Students

Organizers

Events

Registrations

Attendance

Student CRUD

Registrations

Student event registration information

Registration ID	Student	Event	Registration Date
301	Amit Sharma	Coding Competition	2026-09-20
302	Rahul Patil	Coding Competition	2026-09-21
303	Priya Singh	Dance Competition	2026-09-22
304	Sneha Joshi	Cricket Tournament	2026-09-23
305	Rohan Verma	Quiz Competition	2026-09-24

College Event Management System • DBMS Mini Project

College Event Management System
DBMS Mini Project • MySQL

Admin Dashboard

Dashboard

Students

Organizers

Events

Registrations

Attendance

Student CRUD

Attendance

Attendance generated from registrations

Attendance ID	Registration ID	Status
401	301	Present
402	302	Present
403	303	Absent
404	304	Present
405	305	Present

College Event Management System • DBMS Mini Project

College Event Management System
DBMS Mini Project • MySQL

Admin Dashboard

Dashboard

Students

Organizers

Events

Registrations

Attendance

Student CRUD

Student CRUD Operations

Frontend demonstration of Create, Read, Update and Delete

The PDF demonstrates CRUD using Student_ID 106, Karan Mehta. The buttons below simulate those database operations in the browser.

Create / Update Student

Student ID
102

Student Name
Soham Bhisma

Branch
IT

Year
3

Email
sohambhishma@gmail.com

CREATE

UPDATE

CLEAR

College Event Management System
DBMS Mini Project • MySQL

Admin Dashboard

Dashboard

Students

Organizers

Events

Registrations

Attendance

Student CRUD

Student CRUD Operations

Frontend demonstration of Create, Read, Update and Delete

The PDF demonstrates CRUD using Student_ID 106, Karan Mehta. The buttons below simulate those database operations in the browser.

Create / Update Student

Student ID
102

Student Name
Soham Bhisma

Branch
IT

Year
3

Email
sohambhishma@gmail.com

CREATE

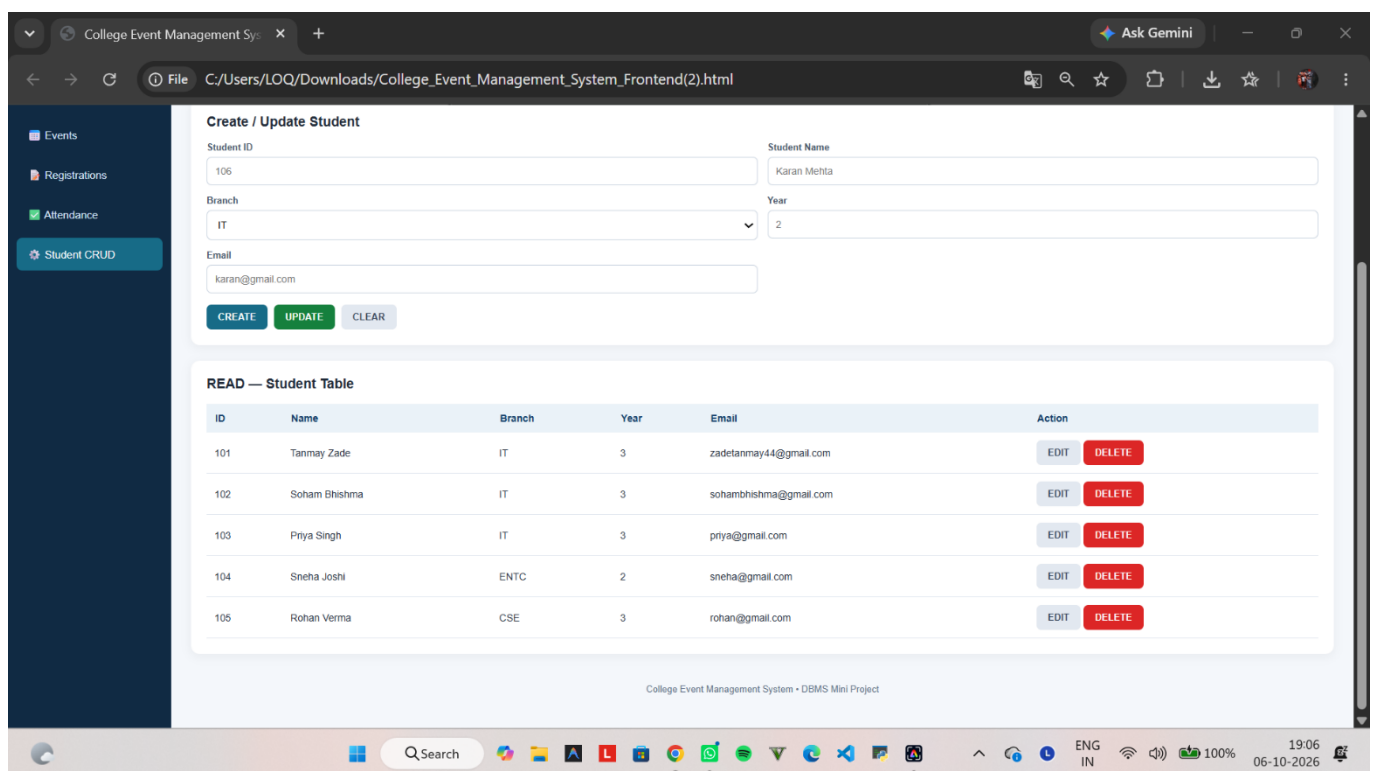
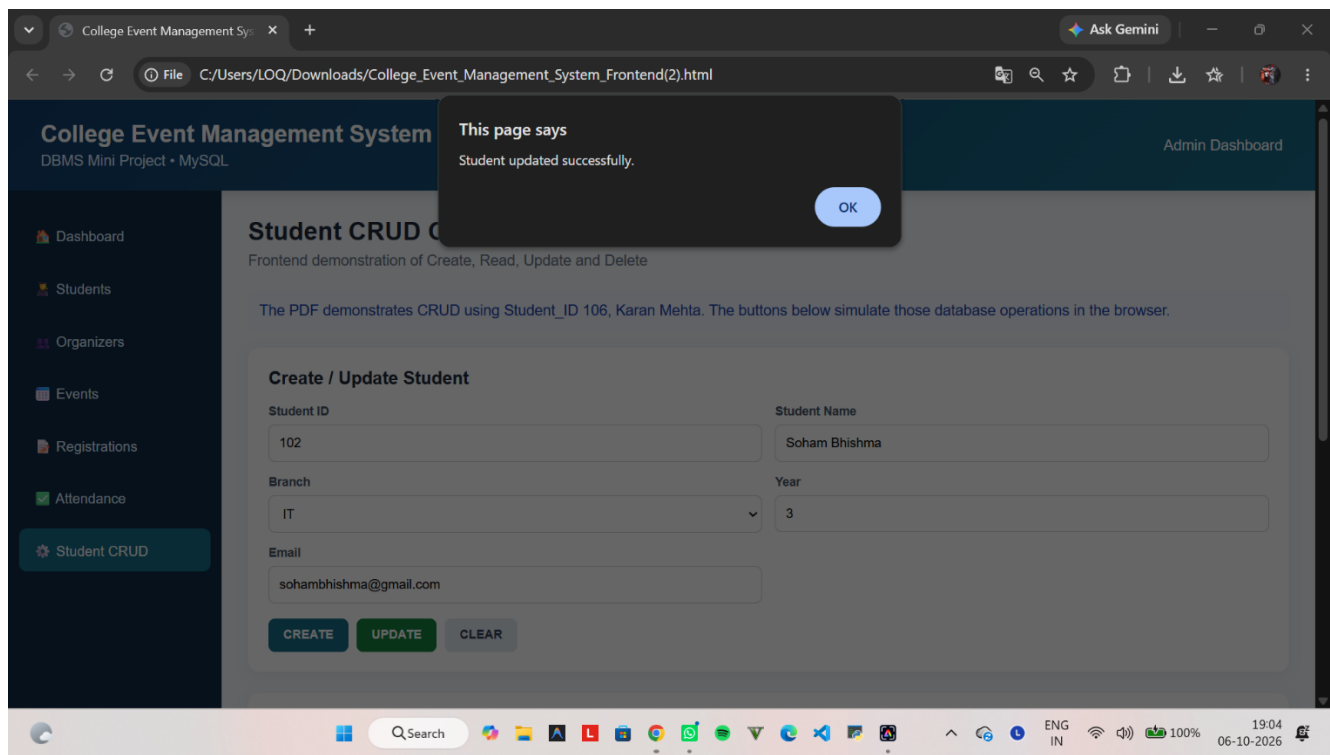
UPDATE

CLEAR

This page says
Student ID already exists.

OK

Page 19



13.RESULT

The College Event Management System was successfully implemented in MySQL. The project demonstrates table creation, data insertion, basic queries, joins, aggregate functions, GROUP BY and HAVING, subqueries, views, indexes, triggers and complete CRUD operations.

14.CONCLUSION

This mini project provides practical understanding of relational database design and SQL operations. It demonstrates how related tables can be used to manage college events and how database features improve data organization and control.

15.ADVANTAGES

- Organized and centralized event information.
- Reduces duplicate data through relational design.
- Faster searching using SQL queries and indexes.
- Automatic attendance creation using a trigger.
- Supports complete CRUD operations.

16.DISADVANTAGES

- Requires database administration and maintenance.
- Incorrect SQL operations can affect stored data.
- The current project is a basic academic implementation and may need additional security and validation.

17.FUTURE SCOPE

- Add a web or mobile interface.
- Add login and role-based access control.
- Add online event registration and notifications.
- Generate attendance and event reports automatically.
- Add certificate generation and other college-event services.